

AnchorNotes Table of Contents

1. Project Title and Authors	1
2. Preface	2
3. Introduction	2
a. Description	2
b. Features	2
i. Note Storage	2
ii. Note Creation	2
iii. Note Search	2
iv. Note Organization	2
v. Built-In Reminders	3
vi. Note Templates	3
4. Architectural Design	3
a. Block Diagram	3
b. High-level architecture (overview)	3
c. Architectural patterns & rationale	4
d. Major components (with responsibilities)	5
e. Key data model (high-level)	6
f. Error handling & offline behavior (architecture-level)	6
g. Component ↔ requirement mapping (traceability)	7
h. External libraries & Android APIs (anticipated)	7
5. Detailed Design	8
a. Class Diagram	8
b. Sequence Diagram — Create Note with Photo and Time Reminder	10
c. Sequence Diagram — Create Note with Location & Geofenced Reminder	11
d. Sequence Diagram — Note Search with Filters and Open Result	13
e. Sequence Diagram — Pin/Unpin Note and Refresh Pinned Section	15
f. Sequence Diagram — Map View: Show Note Pins and Open from Map	17

1. Project Title and Authors

- a. Team 33
- b. The Wanginators
- c. Names and emails:
 - i. Anthony Tran: atran881@usc.edu
 - ii. Arsalan Ghogari: ghogari@usc.edu
 - iii. Himanshu Jain: jainhima@usc.edu
 - iv. Miheer Tongaonkar: mtongaon@usc.edu
 - v. Rishi Jain: rishij@usc.edu

2. Preface

- a. This document focuses on the analysis and design of AnchorNotes. It consolidates the prior requirements outlined in the 2.1 document into a concrete software design that third-party engineers can implement (expressed in standard UML). We present a high-level architecture describing the major components of AnchorNotes' system, as well as their interactions which is shown in a block diagram. This is followed by an implementable detailed design (in the form of a class diagram + sequence diagrams). Additionally, any planned external frameworks or libraries are identified where relevant to the design.

3. Introduction

a. Description

- i. From students to professionals, to anywhere in between, collecting information down is an essential part of life. AnchorNotes allows notetaking to be performed using text, photos, and videos. AnchorNotes serves as a hub for all forms of notetaking for daily life.

b. Features:

- i. Note Storage:
 - 1. Users can create and store as many notes as they want up to their local or cloud storage limit.
- ii. Note Creation:
 - 1. New notes are created by pressing the "New Note" button on the home screen. Users can add voice memos, images, and text to their notes and format text with different font sizes and styles.
- iii. Note Search:
 - 1. Users can quickly locate specific notes by searching through keywords found in titles or note bodies. Searching is also refined using filters like tags, creation/edit dates, or attached media (photos, videos, voice memos, and/or location data).

This combination of keyword search and smart filtering ensures that important information remains organized, discoverable, and accessible whenever needed.

iv. Note Organization:

1. Users can pin, tag, or attach locations to notes. Pinning notes places a note at the top of the home screen regardless of the time it was last modified. Tagging notes allows for easier search, serving as a filter for notes. Attaching a location to notes provides additional context for notes and also enables the geofencing-based reminder feature.

v. Built-In Reminders:

1. Users can choose to receive a notification for specific notes based on entering a certain location (geofencing) or at a certain time.

vi. Note Templates:

1. Users can create, edit, and manage note templates (with page color, pre-filled text, sections, and formatting), which are stored in a list alongside a default example template.
2. Templates can be associated with tags or geofences, and when creating a new note, users can select from available templates, with location-relevant ones prioritized at the top of the list.

4. Architectural Design

a. Block Diagram

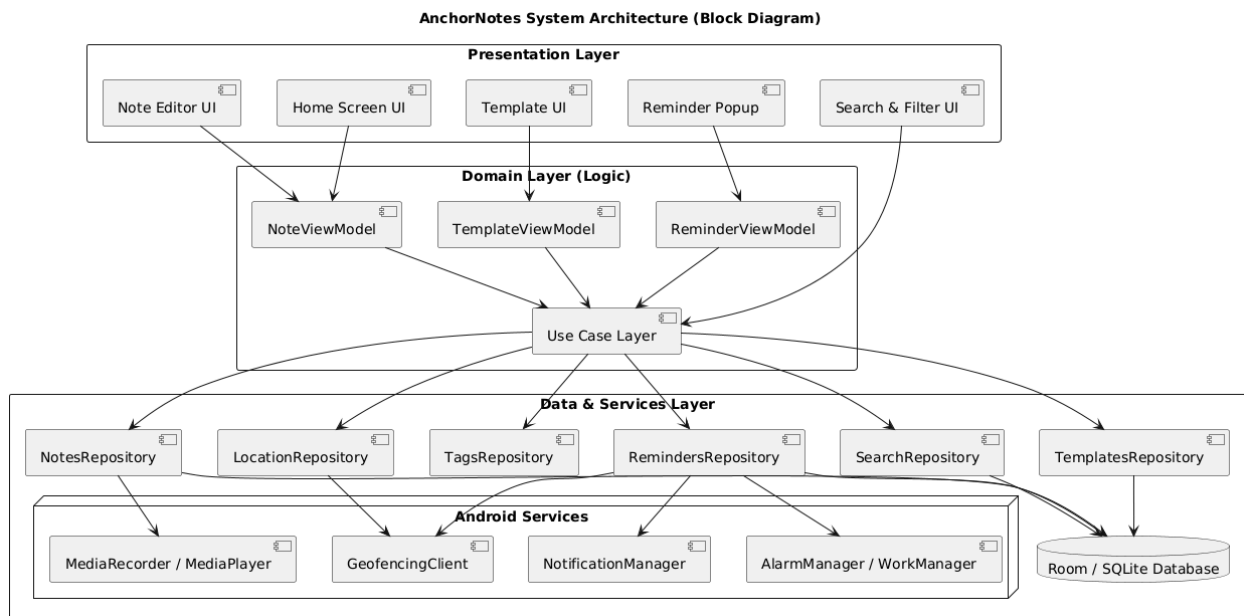


Figure 4.1 – Block Diagram for AnchorNotes

b. High-level architecture (overview)

- AnchorNotes is a modular Android app organized into three layers following MVVM + Repository with an event-driven reminders subsystem:
 1. **Presentation layer (UI)**
 - Screens: Home (Notes list + Pinned + Relevant Notes), Note Editor, Search & Filters, Templates, Map (optional), Settings.
 - Technology: Fragment/Compose views + Navigation; observes ViewModels (LiveData/Flow).
 2. **Domain layer (ViewModels & Use-Cases)**
 - ViewModels coordinate UI state and invoke use-cases.
 - Use-cases encapsulate app logic (CreateNote, AttachMedia, TogglePin, AddTag, SetTimeReminder, SetGeoReminder, RunSearch, GenerateRelevantNotes, CreateTemplate, etc.).
 3. **Data & Services layer**
 - **Repositories:** NotesRepository, TagsRepository, TemplatesRepository, RemindersRepository, SearchRepository, LocationRepository.
 - **Local storage:** Room/SQL entities for Note, Tag, NoteTagCrossRef, Attachment (photo/voice), Reminder (time or geo, mutually exclusive), Template, Location.
 - **Android services:** Foreground GeofencingClient/Location, AlarmManager/WorkManager (time notifications), NotificationManager, MediaRecorder/MediaPlayer, Camera/Photo Picker.
 - **Indexing:** FTS (SQLite) for fast title/body search; metadata columns for tag(s), date range, hasPhoto/hasVoice/hasLocation flags.
- **Runtime connectors:**
 - UI ↔ ViewModels (observer pattern)
 - ViewModels ↔ Use-cases (method calls)
 - Use-cases ↔ Repositories (synchronous/Coroutine calls)

- Repositories ↔ Room DB and Android services (Location, Alarms, Notifications)
- Event bus (SharedFlow): reminder triggers → “Relevant Notes” updates.

c. Architectural patterns & rationale

- **MVVM + Repository**: standard Android architecture that isolates UI from data sources; simplifies unit testing, supports lifecycle awareness, and clean error handling.
- **Event-driven / pub-sub** for reminders: geofence & time alarms publish events consumed by the Relevant Notes Engine; this minimizes coupling and ensures triggers work while the app is backgrounded.
- **Strategy pattern (ReminderStrategy)**: enforces one reminder type per note (time or geofence) via pluggable strategies with validation.
- **Decorator/Composite for attachments**: attachments (photo/voice) extend a note without changing core note logic.
- **Repository facade** around Room + Android services: single place to handle offline, retries, and permission states.
- **FTS index**: supports fast search across large note bodies; filter flags avoid scanning blobs.

d. Major components (with responsibilities)

1. HomeModule

- Displays: All Notes (last-edited sort), **Pinned** section, **Relevant Notes** section.
- Interacts with SearchBar, Tag chips, Filter sheet.

2. NoteEditorModule

- Rich text (bold/italic/font sizes, checklists), photo attach, voice memo record/playback, location attach/update/remove, pin toggle, tag add/remove, autosave.

3. SearchModule

- Keyword search (FTS) + filters (tags, date range, hasPhoto/hasVoice/hasLocation).
- Returns ranked result set to Home.

4. OrganizationModule

- Tag CRUD, pin/unpin; exposes tag suggestions and pinned list maintenance.

5. RemindersModule

- TimeReminderEngine (AlarmManager/WorkManager) and GeoReminderEngine (GeofencingClient).
- Emits ReminderEvents to Relevant Notes Engine; posts notifications; handles removal/cancellation.

6. RelevantNotesEngine

- Subscribes to ReminderEvents; inserts/removes entries:
 - Time-based → auto-expire after 1 hour.
 - Geofence-based → remove on exit event.

7. TemplatesModule

- Template CRUD (page color, pre-filled text/formatting, optional tag(s), optional geofence).
- Template picker prioritizes geofence-relevant templates.

8. MapModule (optional)

- Visualizes notes with locations; supports same search/filters; opens note preview.

9. Persistence & Indexing

- Room DB with DAOs; FTS tables for title/body; derived columns for filters.

10. Platform Services Adapter

- Location/Geofence, Alarms/WorkManager, Notifications, Media, Permissions.

e. Key data model (high-level)

- **Note**(id, title, body, lastEditedAt, isPinned, hasPhoto, hasVoice, locationId?, templateId?)
- **Attachment**(id, noteId, type {PHOTO|VOICE}, uri, duration?)
- **Tag**(id, name), **NoteTagCrossRef**(noteId, tagId)
- **Location**(id, lat, lon, radius?, placeName?)
- **Reminder**(id, noteId, type {TIME|GEO}, triggerAt?, geold?, status {ACTIVE|CANCELLED|FIRED})
- **Template**(id, name, pageColor, prefilledBody, defaultTags[], optionalGeold?)

f. Error handling & offline behavior (architecture-level)

- **Location attached while offline/no GPS:** LocationRepository returns LocationUnavailable; UI shows non-blocking message and saves note without location; user can retry later.

- **Enter geofence while offline:** event is ignored (design requirement); next location update outside geofence won't retro-trigger.
- **Remove reminder:** `RemindersRepository.cancel(notelId)` cancels alarms/geofences and deletes the Reminder row; Relevant Notes entry removed if present.
- **Single reminder enforcement:** `ReminderStrategy.setReminder(note)` checks existing type; if conflict, returns `ValidationError` and shows UI error.
- **Autosave:** Editor emits debounced `SaveNote` use-case calls; crashes/app restarts reopen last state from DB.
- **Permissions:** centralized `PermissionsManager`; graceful degradation (no location features without runtime grant).

g. Component ↔ requirement mapping (traceability)

- **Note creation & storage** → `NoteEditorModule`, `HomeModule`, `Persistence` (text, images, voice, checklists, autosave).
- **Pinned section / pin & unpin** → `HomeModule` + `OrganizationModule`.
- **Tags (create/add/remove)** → `OrganizationModule` + `Persistence`.
- **Location add/view/update/remove** → `NoteEditorModule` + `LocationRepository` + `MapModule` (optional view).
- **Search & filter (tags/date/media/location)** → `SearchModule` + `FTS` + derived flags.
- **Reminders (time / geofence; one type only)** → `RemindersModule` + `Strategy` + `RelevantNotesEngine` + `Notifications`.
- **Relevant Notes dwell rules** → `RelevantNotesEngine` (1-hour expiry for time; remove on exit for geofence).
- **Templates (CRUD, tag/geofence assoc, prioritized by geofence)** → `TemplatesModule` + Picker logic.
- **Note Map (optional)** → `MapModule`.

h. External libraries & Android APIs (anticipated)

- Room / SQLite** — local storage & indexing.
- Coroutines / Flow** — async & event streams.
- WorkManager / AlarmManager** — time-based reminders (reliable, doze-aware).
- FusedLocationProvider + GeofencingClient** — geofence attach & triggers.
- NotificationCompat** — lock-screen & heads-up notifications.

- vi. **MediaRecorder/MediaPlayer & Photo Picker** — voice/photo attachments.
- vii. **Maps SDK** — Note Map visualization.

5. Detailed Design

a. Class Diagram:

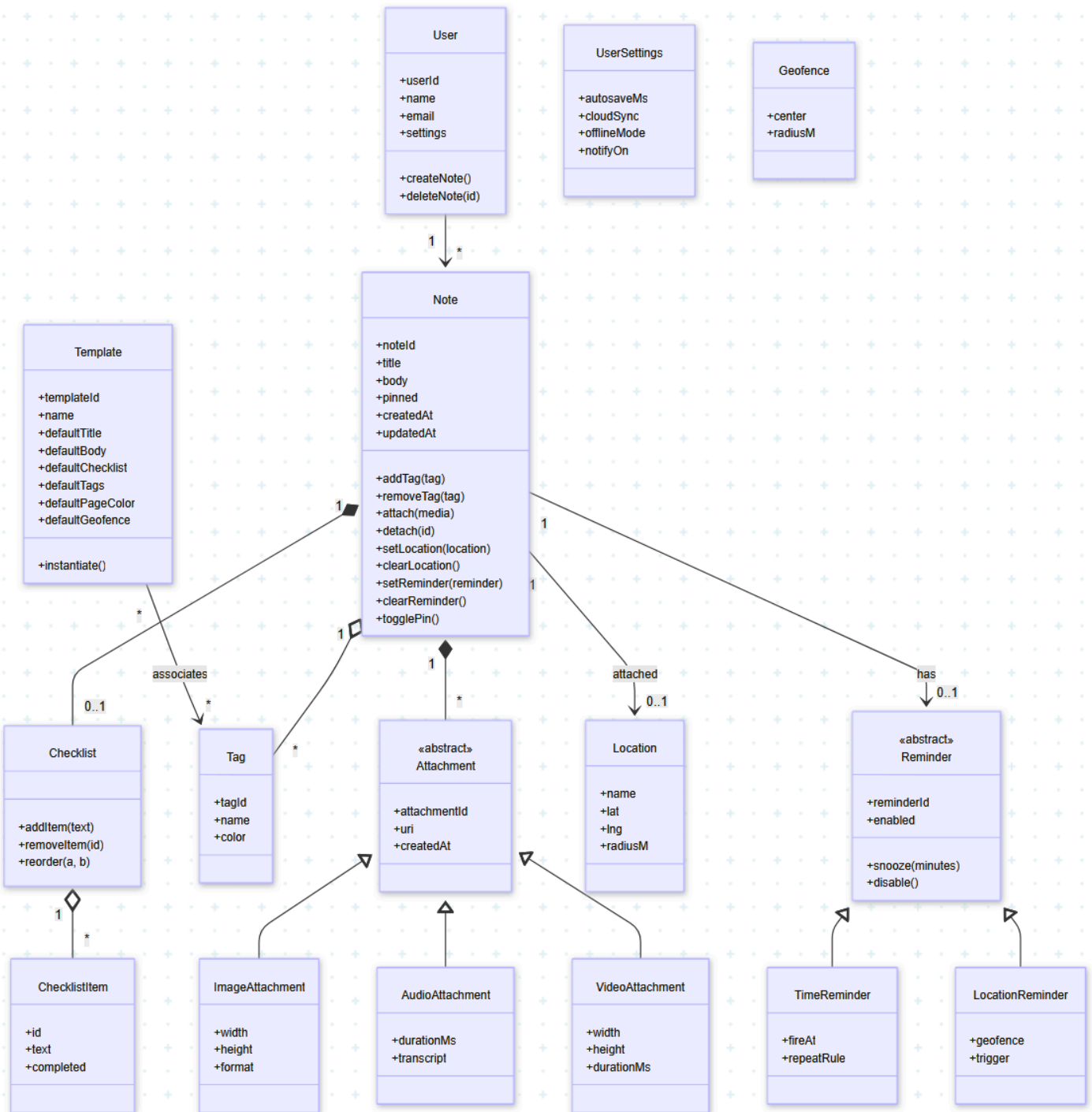


Figure 5.1 – Class diagram for AnchorNotes showing class relations. Below the name of each class is 2 halves: top half for members, bottom half for methods.

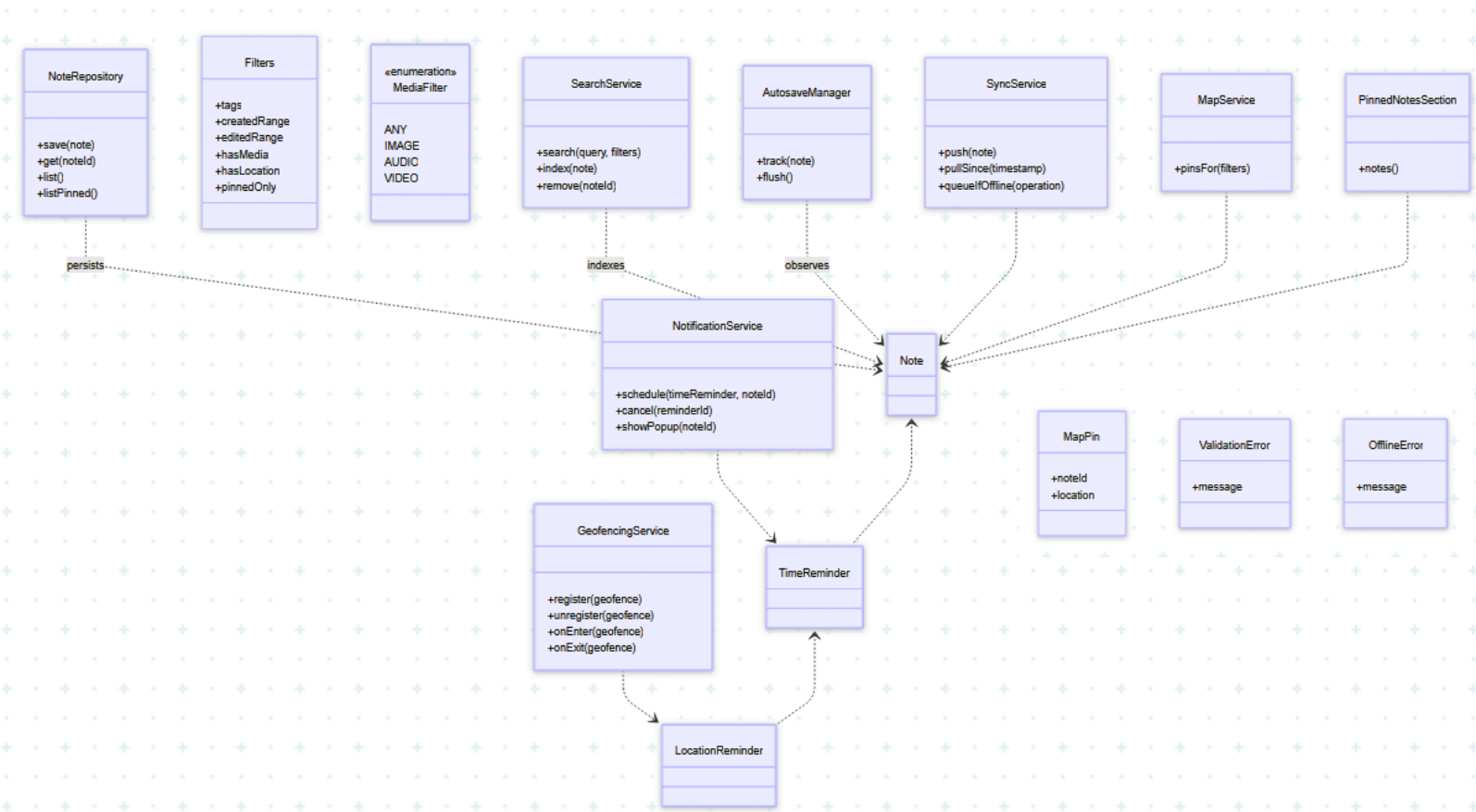


Figure 5.2 — Class diagram for AnchorNotes showing services. Below the name of each class is 2 halves: top half for members, bottom half for methods.

b. Sequence Diagram — Create Note with Photo and Time Reminder

i. Purpose

1. This diagram models the process of creating a new note with an attached photo and a time-based reminder.
2. It shows how the user interacts with the system and how objects such as Note, Attachment, Reminder, and the NotesRepository collaborate to store data and schedule notifications.

ii. Diagram

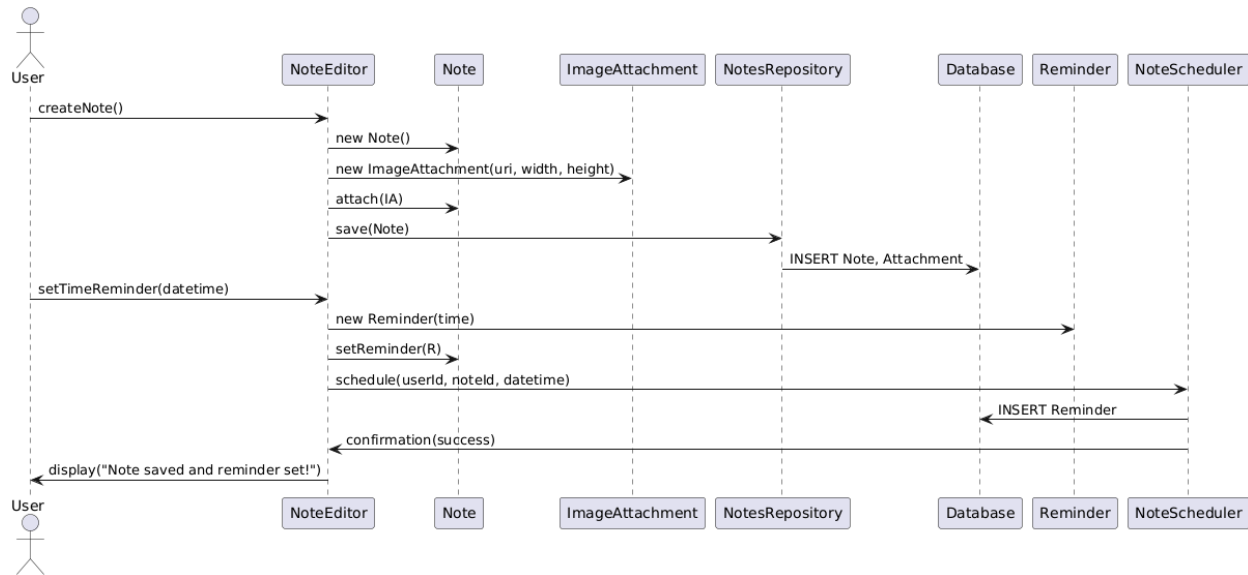


Figure 5.3 – Sequence diagram for “Create Note with Photo and Time Reminder.”

iii. Description of Interactions

1. User → NoteEditor

- a. The user initiates creation of a new note from the UI.

2. NoteEditor → Note

- a. A new Note object is instantiated.

3. NoteEditor → ImageAttachment

- a. Photo attachment object created with metadata (URI, width, height).

4. NoteEditor → Note

- a. Attachment is linked to the note.

5. NoteEditor → NotesRepository

- a. Note and its attachments are sent for persistence.

6. NotesRepository → Database

- a. Note and photo records are stored.

7. User → NoteEditor

- a. User adds a time-based reminder.

8. NoteEditor → Reminder / Note / NoteScheduler

- a. Reminder object created, associated with note, and scheduled.

9. NoteScheduler → Database → NoteEditor

- a. Reminder record persisted and confirmation returned.

10. NoteEditor → User

- a. UI confirms note creation and reminder setup.

iv. Postconditions

- The note (with photo) is stored in the local database.
- The time-based reminder is successfully registered with the NoteScheduler.
- The note will appear in the **Relevant Notes** section once the reminder triggers.

c. Sequence Diagram – Create Note with Location & Geofenced Reminder

i. Purpose

- This sequence diagram demonstrates how the app's components work together to create notes, attach locations, set a geofenced reminder, handle online/offline registration, and deliver the notification to the user when entering an area.

ii. Diagram

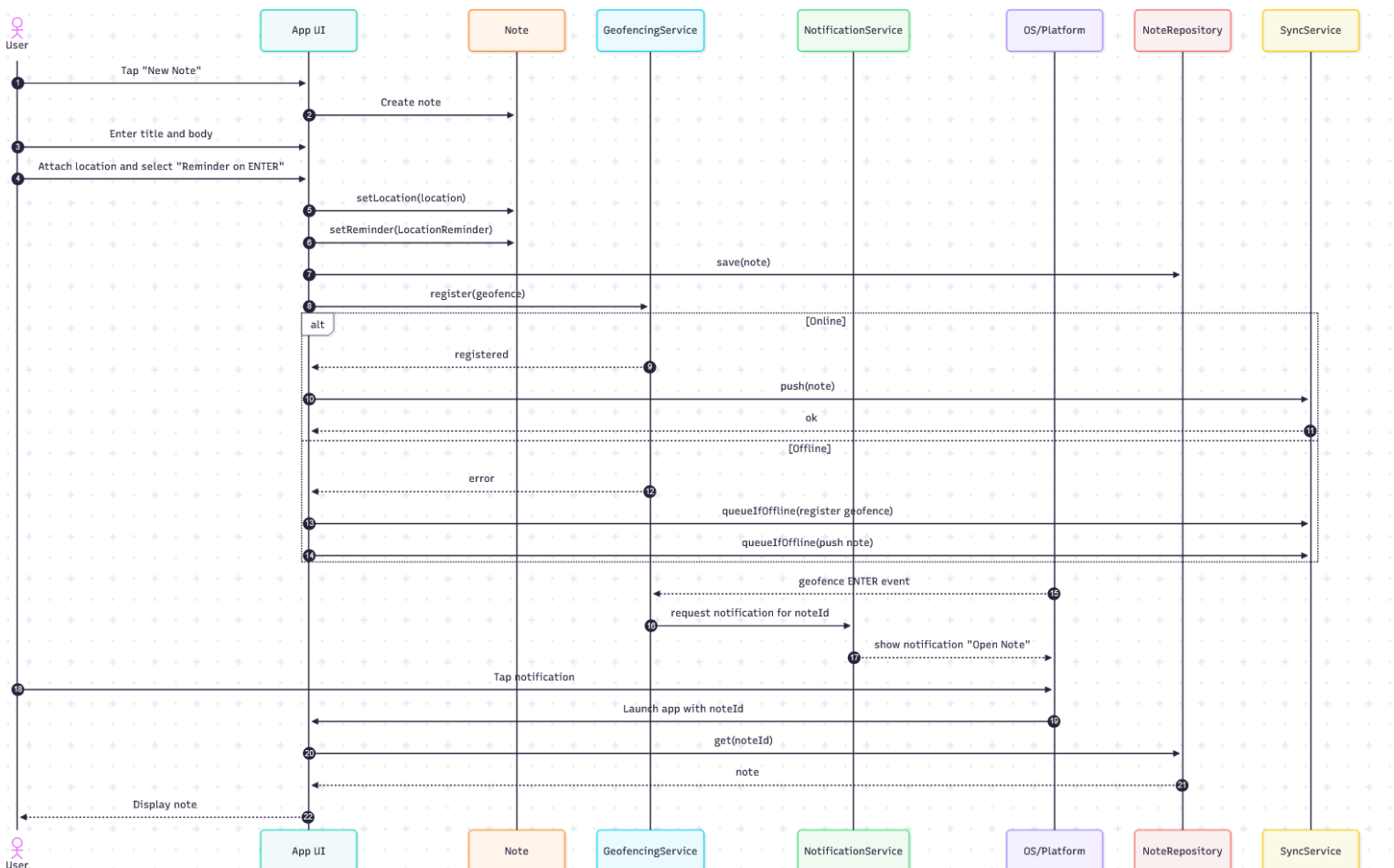


Figure 5.4 — Sequence diagram for “Create Note with Location & Geofenced Reminder.”

iii. Description of Interactions

1. User → App UI

- a. User taps “New Note” to start creation

2. App UI → Note

- a. A new Note object is instantiated.

3. User → App UI

- a. User enters title/body content.
- b. User attaches a location and chooses “Reminder on ENTER”.

4. App UI → Note

- a. setLocation(location) associates the chosen place with the note.
- b. setReminder(LocationReminder) attempts to set a geofenced reminder.

5. Note → App UI

- a. If a reminder already exists, the note rejects the second one

6. App UI → NoteRepository

- a. save(note) persists the note and its updated metadata.

7. App UI → GeofencingService

- a. register(geofence) requests OS-level geofence registration.

8. GeofencingService → App UI (Online branch)

- a. Returns “registered”. UI optionally calls SyncService to sync remote state.

9. GeofencingService → App UI (Offline branch)

- a. Returns an error (offline)

10. OS/Platform → GeofencingService (Later, at runtime)

- a. Emits a geofence ENTER event when the device enters the defined area.

11. GeofencingService → NotificationService

- a. Requests delivery of a notification for the associated noteld.

12. NotificationService → OS/Platform

- a. Shows system notification (“Open Note”).

13. User → OS/Platform

- a. User taps the notification.

14. OS/Platform → App UI

- a. Launches the app linked to the noteld.

15. App UI → NoteRepository

- a. get(noteld) loads the latest copy of the note.

16. NoteRepository → App UI

- a. Returns the note data.

17. App UI → User

- a. Displays the note screen (focuses it in Relevant Notes).

iv. Postconditions

1. A new Note exists with a saved title/body and it's persisted in NoteRepository.
2. The note has exactly one reminder set. If another existed, it was rejected or replaced.
3. The note has an associated location/geofence
4. **Online:** The geofence is registered with the OS via GeofencingService, and the latest note state is synced.
5. **Offline:** Geofence registration and sync are queued. No geofence trigger will fire until registration succeeds at a later point.
6. When the user later enters the geofence, a notification is delivered. Tapping it loads up the app and displays the note.

d. Sequence Diagram – Note Search with Filters and Open Result

i. Purpose

1. This diagram models the process of searching for a note with filters, and opening a selected note.
2. It shows how the user interacts with the UI and how the objects and the system aspects such as the SearchService, NotesRepository and the database all interact with each other to query and display results.

ii. Diagram

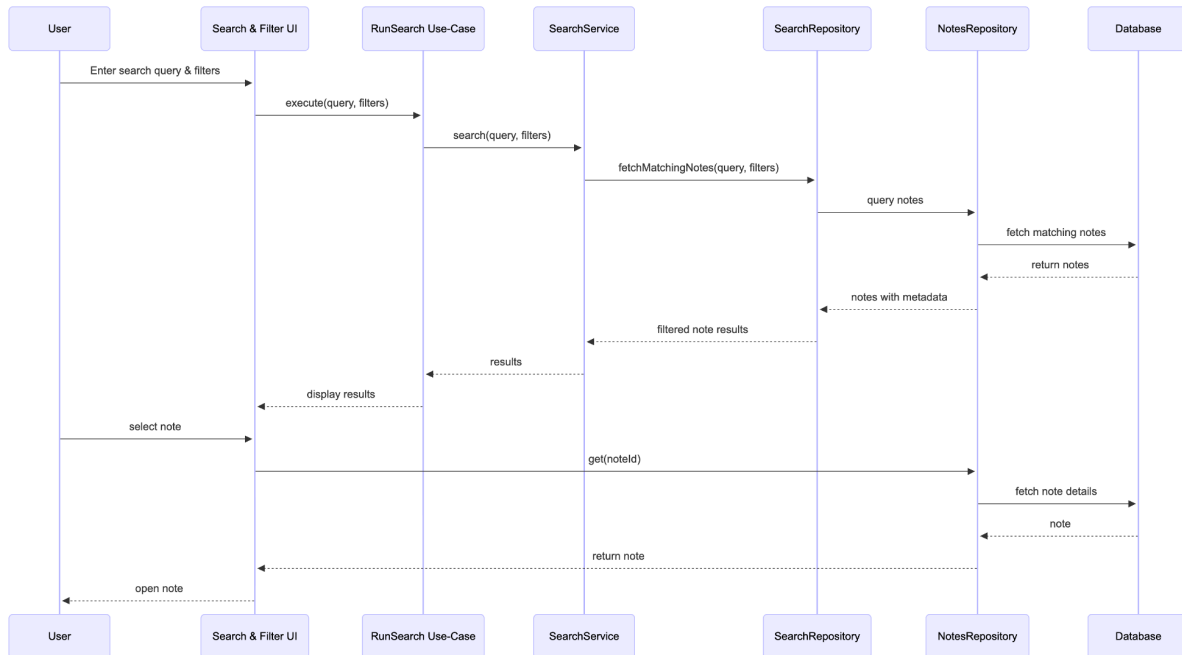


Figure 5.5 — Sequence diagram for “Note Search with Filters and Open Result”

iii. Description of Interactions

1. User → Search & Filter UI

- a. The user enters a search query and filters

2. Search UI → RunSearch UseCase

- a. The UI calls the RunSearch use-case with the query and the filters

3. RunSearch → SearchService

- a. The search is delegated to SearchService

4. SearchService → SearchRepository

- a. SearchService queries the SearchRepository

5. SearchRepository → NotesRepository

- a. The SearchRepository accesses the main NotesRepository for note data

6. NotesRepository → Database

- a. The repository queries the database

7. Database → NotesRepository

- a. Returns the notes that match the query

8. NotesRepository → SearchRepository

- a. Returns notes with necessary metadata

9. SearchRepository → SearchService

- a. Returns the filtered note results

10. SearchService → RunSearch

- a. Sends the results back to the use-case

11. RunSearch → Search UI

- a. Updates the UI to display the result

12. User → Search UI

- a. A note is selected

13. Search UI → NotesRepository

- a. Requests the full note details and content

14. NotesRepository → Database

- a. Fetches the note details

15. Database → NotesRepository

- a. Note is returned

16. NotesRepository → Search UI

- a. Note is returned to the UI

17. Search UI → User

- a. Opens and displays the selected note

iv. Postconditions

- The search results are displayed to the user according to the query and applied filters
- The selected note is opened with the most updated data and information
- The note can be edited (and can be interacted with like normal) from the note detail screen

e. Sequence Diagram – Pin/Unpin Note and Refresh Pinned Section

i. Purpose:

1. This diagram models the process of pinning or unpinning a note and refreshing the pinned notes section displayed on the Home screen.
2. It shows how the user interacts with the UI, how the NoteEditor or HomeModule communicates with the NotesRepository, and how the pinned notes list is dynamically refreshed to reflect the user's action.
3. This process ensures that pinned notes always appear at the top of the Home screen in real-time, maintaining user convenience and organization.

ii. Diagram

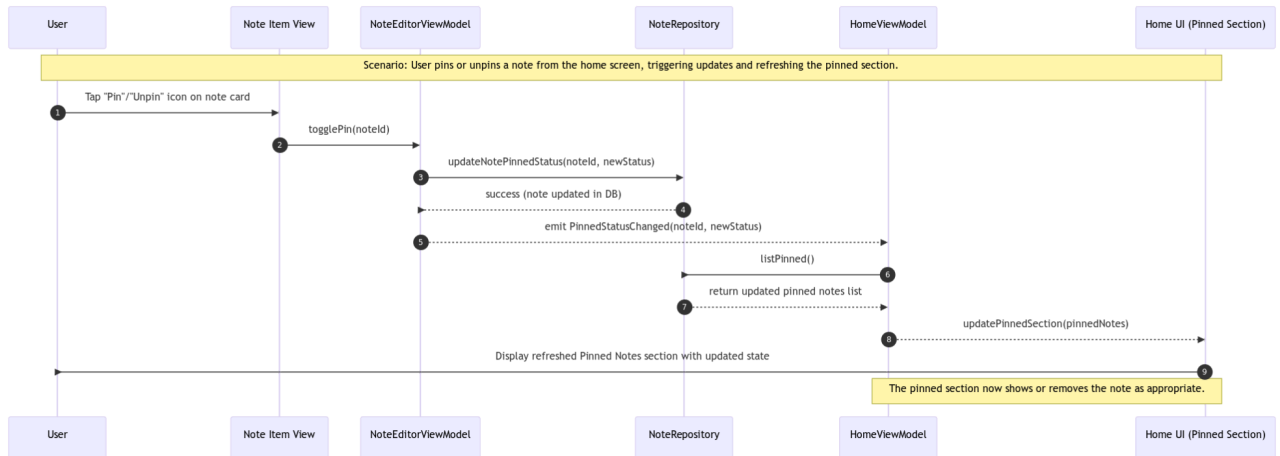


Figure 5.6 – Sequence diagram for “Pin/Unpin Note and Refresh Pinned Section”

iii. Description of Interactions

1. User → HomeUI

- a. The user taps the pin icon next to a note in the Home screen or within the Note Editor to pin or unpin it.

2. HomeUI → NoteEditor

- a. The UI delegates this action to the NoteEditor, which manages the note’s editable state and properties.

3. NoteEditor → Note

- a. The togglePin() method is called on the note object, switching its isPinned attribute between true and false.

4. Note → NoteEditor

- a. The updated note object returns to the editor, reflecting its new pinned/unpinned state.

5. NoteEditor → NotesRepository

- a. The editor requests the repository to persist the change by saving the updated note.

6. NotesRepository → Database

- a. The repository updates the isPinned field in the Note table within the Room database.

7. Database → NotesRepository → NoteEditor

- a. The database confirms the update, and the repository acknowledges successful persistence to the NoteEditor.

8. NoteEditor → HomeUI

- a. The editor emits an event or callback notifying the UI that the note has been updated.

9. HomeUI → PinnedSection

- a. The UI triggers a refresh of the Pinned Notes section to reflect the latest pinned/unpinned state.

10. PinnedSection → NotesRepository → Database

- a. The pinned section queries all notes where isPinned = true to update the list displayed at the top of the screen.

11. Database → NotesRepository → PinnedSection → HomeUI

- a. The query results are returned, and the pinned section updates the Home screen view accordingly.

12. HomeUI → User

- a. The user sees the refreshed note order: pinned notes remain fixed at the top, while unpinned notes move back to the normal chronological list.

iv. Postconditions

1. The note's pinned status is correctly updated and persisted in the local database.
2. The Home screen immediately reflects the change in the Pinned section.
3. Any subsequent searches or filters also respect the updated isPinned attribute.
4. The user experiences a seamless real-time refresh, maintaining consistency between stored data and displayed content.

f. Sequence Diagram — Map View: Show Note Pins and Open from Map

i. Purpose

1. This diagram models the process of how a user interacts with the Note Map feature to view and access location-based notes. It shows how notes with attached locations are retrieved and displayed as pins on the map, how users apply filters such as tags, date, and media type, and how tapping on a pin displays a preview with the option to open the full note.
2. The purpose of this feature is to provide a spatial view of notes, allowing users to quickly locate and revisit information based on where it was created or is most relevant.

ii. Diagram

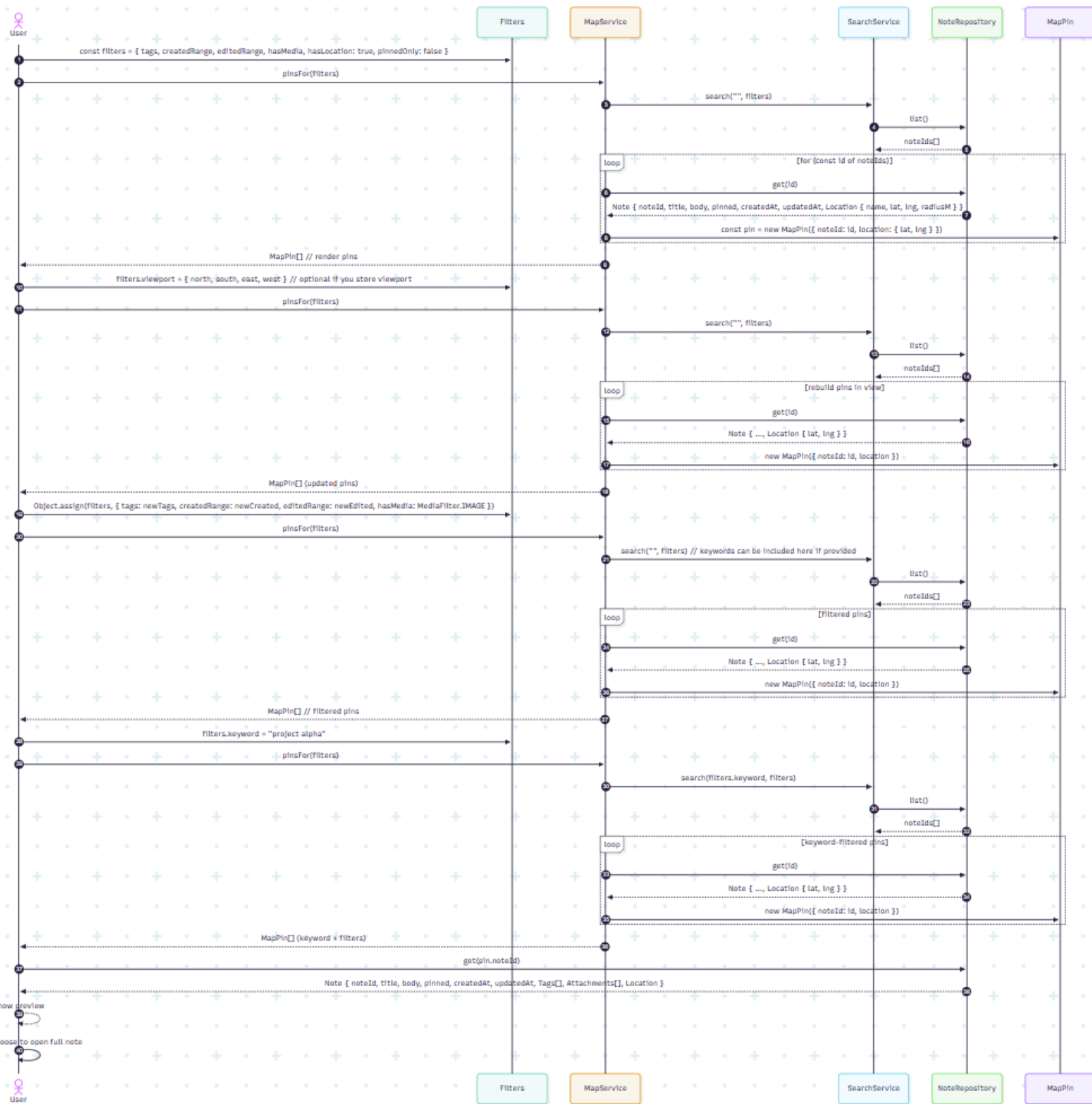


Figure 5.7 — Sequence diagram for “Map View: Show Note Pins and Open from Map”

iii. Description of Interactions

1. User → Filters

- a. User instantiates and populates Filters with members: tags, createdRange, editedRange, hasMedia, hasLocation = true, pinnedOnly = false.

2. User → MapService

- a. User calls pinsFor(filters) to request pins for the current map view.

3. MapService → SearchService

- a. search(query="", filters) is invoked to find notes that satisfy the Filters members, including hasLocation when set.

4. SearchService → NoteRepository

- a. Retrieves candidates via repository calls such as list() or previously indexed references.
- b. Returns notelds[] that match tags, createdRange, editedRange, hasMedia, pinnedOnly, and hasLocation.

5. MapService → NoteRepository

- a. Iterates over notelds[] and calls get(noteld) for each to obtain Note with its attached Location.
- b. Uses Location members name, lat, lng, and radius.

6. MapService → MapPin

- a. For each note that has a Location, constructs a MapPin with members noteld and location.

7. MapService → User

- a. Returns the array MapPin[] for rendering.

8. User → Filters

- a. User modifies any of the Filters members (tags, createdRange, editedRange, hasMedia, pinnedOnly). hasLocation may remain true for the map.

9. User → MapService

- a. Calls pinsFor(filters) again. Steps 3 through 7 repeat to refresh the visible pins.

10. User → MapService

- a. On pin tap, user asks for the note tied to that pin using pin.noteld.

11. MapService → NoteRepository

- a. Calls `get(pin.noteId)` to fetch the Note aggregate, including title, body, pinned, createdAt, updatedAt, and any Tag or Attachment as needed for a preview.

12.MapService → User

- a. Returns the Note data to the UI so the user can preview or open the note.

iv. Postconditions

1. All notes that contain valid Location data and satisfy the selected Filters are displayed as pins on the map.
2. Each MapPin corresponds to a specific Note object, linking its noteId and Location attributes.
3. Adjusting filters or zoom level dynamically updates visible pins using `MapService.pinsFor(filters)` without requiring a full refresh.
4. Tapping a pin displays a note preview, showing key fields such as title, body snippet, and updatedAt.
5. Selecting Open Note retrieves the complete Note from `NoteRepository.get(noteId)` and navigates the user to the note view or editor.
6. If no notes match the criteria, the system displays an empty map state.
7. If location permissions are disabled, the map remains accessible but without note pins or geolocation filters.
8. All map interactions (filtering, zooming, tapping) execute without data loss or disruption to other application modules.